Python Programming - 2301CS404

Lab - 15 (1)

Roll No.: 310

Name: Ronit Bhadania

01) Write a Program to create a class by name Students, and initialize attributes like name, age, and grade while creating an object.

In [2]:
```
class Students :
    def __init__(self,name,age,grade) :
        self.name = name
        self.age = age
        self.grade = grade

s1 = Students("Ronit",19,"A")
print(s1.name)
print(s1.age)
print(s1.grade)
```

```
Ronit
19
A
```

02) Create a class named Bank_Account with Account_No, User_Name, Email,Account_Type and Account_Balance data members. Also create a method GetAccountDetails() and DisplayAccountDetails(). Create main method to demonstrate the Bank_Account class.

In [4]:
```
class Bank_Account :
    # def __init__(self,Account_No,User_Name,Email,Account_Type,Account_Balance)
    #     self.Account_No = Account_No
    #     self.User_Name = User_Name
    #     self.Email = Email
    #     self.Account_Type = Account_Type
    #     self.Account_Balance = Account_Balance
```

```
    def GetAccountDetails(self) :
        self.Account_No = int(input("Enter account no.:"))
        self.User_Name = input("Enter your name:")
        self.Email = input("Enter your email:")
        self.Account_Type = input("Enter your accounttype:")
        self.Account_Balance = int(input("Enter the balance:"))

    def DisplayAccountDetails(self) :
        print(self.Account_No)
        print(self.User_Name)
        print(self.Email)
        print(self.Account_Type)
        print(self.Account_Balance)

b1 = Bank_Account()
b1.GetAccountDetails()
b1.DisplayAccountDetails()
```

```
125
Ronit
ronit@gmail.com
saving
10000
```

03) WAP to create Circle class with area and perimeter function to find area and perimeter of circle.

In [6]:
```
import math
class Circle :
    def __init__(self,radius) :
        self.radius = radius

    def area(self) :
        return math.pi * self.radius**2

    def perimeter(self) :
        return 2 * math.pi * self.radius

radius = float(input("Enter the radius of the circle: "))
my_circle = Circle(radius)
print(f"Area of the circle: {my_circle.area():.2f}")
print(f"Perimeter of the circle: {my_circle.perimeter():.2f}")
```

```
Area of the circle: 314.16
Perimeter of the circle: 62.83
```

04) Create a class for employees that includes attributes such as name, age, salary, and methods to update and display employee information.

In [10]:
```
class Employee:
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary

    def update_info(self, name=None, age=None, salary=None):
```

```
        if name: self.name = name
        if age: self.age = age
        if salary: self.salary = salary

    def display_employeeinfo(self):
        print(f"Name: {self.name},Age: {self.age},Salary: {self.salary:,}")

emp1 = Employee("Ronit Bhadania", 30, 75000)
emp1.display_employeeinfo()

emp1.update_info(salary=90000)
emp1.display_employeeinfo()
```

```
Name: Ronit Bhadania,Age: 30,Salary: 75,000
Name: Ronit Bhadania,Age: 30,Salary: 90,000
```

05) Create a bank account class with methods to deposit, withdraw, and check balance.

In [11]:
```
class BankAccount:
    def __init__(self, owner, balance=0.0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited {amount:.2f}. New balance: {self.balance:.2f}")
        else:
            print("Deposit amount must be positive.")

    def withdraw(self, amount):
        if 0 < amount <= self.balance:
            self.balance -= amount
            print(f"Withdrew {amount:.2f}. Remaining balance: {self.balance:.2f}
        elif amount > self.balance:
            print("Insufficient funds.")
        else:
            print("Withdrawal amount must be positive.")

    def check_balance(self):
        print(f"Account Owner: {self.owner} | Current Balance: {self.balance:.2f
        return self.balance

my_account = BankAccount("Ronit", 1000.0)
my_account.deposit(5000)
my_account.withdraw(3000)
my_account.check_balance()
```

```
Deposited 5000.00. New balance: 6000.00
Withdrew 3000.00. Remaining balance: 3000.00
Account Owner: Ronit | Current Balance: 3000.00
```

Out[11]: 3000.0

06) Create a class for managing inventory that includes attributes such as item name, price, quantity, and methods to add, remove, and update items.

In [8]:
```
class InventoryManager:
    def __init__(self):
        self.inventory = {}

    def add_item(self, name, price, quantity):
        if name in self.inventory:
            self.inventory[name]['quantity'] += quantity
        else:
            self.inventory[name] = {'price': price, 'quantity': quantity}
        print(f"Success: Added {quantity} unit(s) of '{name}'.")

    def remove_item(self, name):
        if name in self.inventory:
            del self.inventory[name]
            print(f"Success: '{name}' has been removed.")
        else:
            print(f"Error: '{name}' not found.")

    def update_item(self, name, price=None, quantity=None):
        if name in self.inventory:
            if price is not None:
                self.inventory[name]['price'] = price
            if quantity is not None:
                self.inventory[name]['quantity'] = quantity
            print(f"Success: Updated details for '{name}'.")
        else:
            print(f"Error: '{name}' not found.")

    def display_inventory(self):
        if not self.inventory:
            print("\nInventory is currently empty.")
        else:
            print("\n--- Current Inventory Status ---")
            print(f"{'Item Name':<15} | {'Price':<10} | {'Quantity':<8}")
            print("-" * 40)
            for name, details in self.inventory.items():
                print(f"{name:<15} | {details['price']:<9.2f} | {details['quanti

my_store = InventoryManager()

my_store.add_item("Laptop", 43000.00, 10)
my_store.add_item("Wireless Mouse", 2500.50, 50)
my_store.add_item("Monitor", 30000.00, 15)

my_store.update_item("Laptop", price=40000.00, quantity=8)

my_store.remove_item("Monitor")

my_store.display_inventory()
```

```
Success: Added 10 unit(s) of 'Laptop'.
Success: Added 50 unit(s) of 'Wireless Mouse'.
Success: Added 15 unit(s) of 'Monitor'.
Success: Updated details for 'Laptop'.
Success: 'Monitor' has been removed.

--- Current Inventory Status ---
Item Name       | Price      | Quantity
----------------------------------------
Laptop          | 40000.00   | 8
Wireless Mouse  | 2500.50    | 50
```

07) Create a Class with instance attributes of your choice.

In [12]:
```python
class Student:

    def __init__(self, roll, name, marks):
        self.roll = roll
        self.name = name
        self.marks = marks

s1 = Student(101, "Ronit", 92)

print("Roll No:", s1.roll)
print("Name:", s1.name)
print("Marks:", s1.marks)
```

```
Roll No: 101
Name: Ronit
Marks: 92
```

08) Create one class student_kit

Within the student_kit class create one class attribute principal name (Mr ABC)

Create one attendance method and take input as number of days.

While creating student take input their name .

Create one certificate for each student by taking input of number of days present in class.

In [13]:
```python
class student_kit:

    principal_name = "Mr ABC"

    def __init__(self, name):
        self.name = name

    def attendance(self, days):
        self.days_present = days

    def certificate(self):
        print("\n----- Certificate -----")
        print("Principal:", student_kit.principal_name)
        print("Student Name:", self.name)
        print("Days Present:", self.days_present)
```

```
        print("----------------------")

name = input("Enter student name: ")
s1 = student_kit(name)

days = int(input("Enter number of days present: "))
s1.attendance(days)

s1.certificate()
```

```
----- Certificate -----
Principal: Mr ABC
Student Name: Ronit
Days Present: 310
----------------------
```

09) Define Time class with hour and minute as data member. Also define addition method to add two time objects.

In [14]:

```
class Time:

    def __init__(self, hour, minute):
        self.hour = hour
        self.minute = minute

    def add_time(self, t2):
        h = self.hour + t2.hour
        m = self.minute + t2.minute

        if m >= 60:
            h = h + 1
            m = m - 60

        return Time(h, m)

    def display(self):
        print(self.hour, "hours", self.minute, "minutes")

t1 = Time(2, 50)
t2 = Time(3, 30)
t3 = t1.add_time(t2)

print("Time 1:")
t1.display()

print("Time 2:")
t2.display()

print("Total Time:")
t3.display()
```

```
Time 1:
2 hours 50 minutes
Time 2:
3 hours 30 minutes
Total Time:
6 hours 20 minutes
```